

CampusConnect

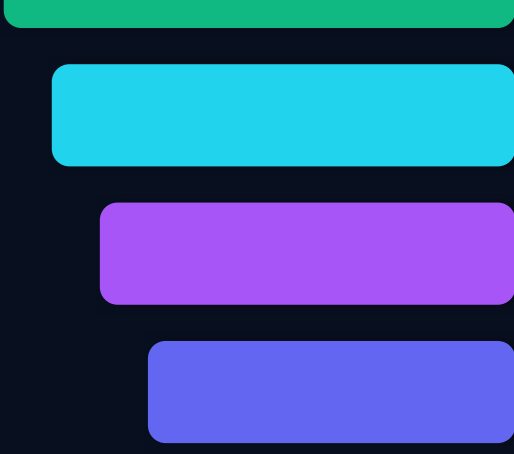
Unified Academic Intelligence & Role-Based Analytics Platform

Project analysis based on the uploaded React/FastAPI codebase and the deployed frontend.

Built with

React + Vite

**FastAPI + JWT + Pandas + Excel +
Recharts**



1. Project Overview

What the application is designed to solve

- Centralizes student, attendance and marks data into one analytics platform.
- Provides different screens and permissions for Faculty, HOD and Principal.
- Stores operational data in branch/semester-wise Excel files.
- Turns raw academic records into KPIs, risk categories, charts and downloadable data.
- Supports CSE, ECE, EEE, MECH and CIVIL across Sem_1, Sem_3, Sem_4 and Sem_5.

Core idea

**Data → API →
Role-based dashboards**

Excel

Academic records

FastAPI

Secure processing

React

Interactive UI

Insights

Risk & decisions

2. System Architecture

Frontend, backend, authentication and data layers



- Frontend calls authenticated API endpoints through a centralized configuration.
- FastAPI enforces JWT authentication, role authorization and branch-level access.
- Pandas loads and merges Students.xlsx, Attendance.xlsx and Mid_Marks.xlsx.
- Analytics modules calculate attendance, marks, risk, fee-pending and explorer data.

3. Technology Stack

What is actually present in the codebase

Frontend

React 19, Vite, custom CSS, Lucide icons, Recharts

Backend

Python, FastAPI, Pydantic

Security

JWT bearer tokens, 120-minute expiry, role checks

Data

Pandas + Excel workbooks organized by branch and semester

Deployment

Vercel frontend configuration + Render deployment guide

4. Data Organization & Files

Branch/semester structure discovered in the uploaded project

backend/Data

- CSE
- ECE
- EEE
- MECH
- CIVIL

Per Branch

- Semester_Sem_1
- Semester_Sem_3
- Semester_Sem_4
- Semester_Sem_5

Per Semester

- Students.xlsx — identity, b section, email, phone
- Attendance.xlsx — month, days, attended days, percent
- Mid_Marks.xlsx — semester-specific subject co and average

5. Login Experience

Deployed frontend + code-based UI analysis

Preset account cards are also included for quick role-based login.

Validation & UX

- Disabled while loading
- Enter key submits
- API error shown in alert card
- JWT validated on app reload



The image shows a login form for 'CampusConnect'. At the top left is a logo of a graduation cap with a yellow tassel. To its right is the text 'CampusConnect' in a large, bold, white font, followed by 'Analytics Platform • Unified Academic Intelligence' in a smaller white font. Below this is a 'Sign In' section. It starts with the text 'Sign In' in bold. Underneath is the label 'Username' followed by a dark blue input field containing the placeholder text 'Enter username'. Below that is the label 'Password' followed by a dark blue input field containing a series of dots. At the bottom of the 'Sign In' section is a large, rounded rectangular button with a blue gradient and the text 'Sign In' in white.

6. Demo Logins & Access Levels

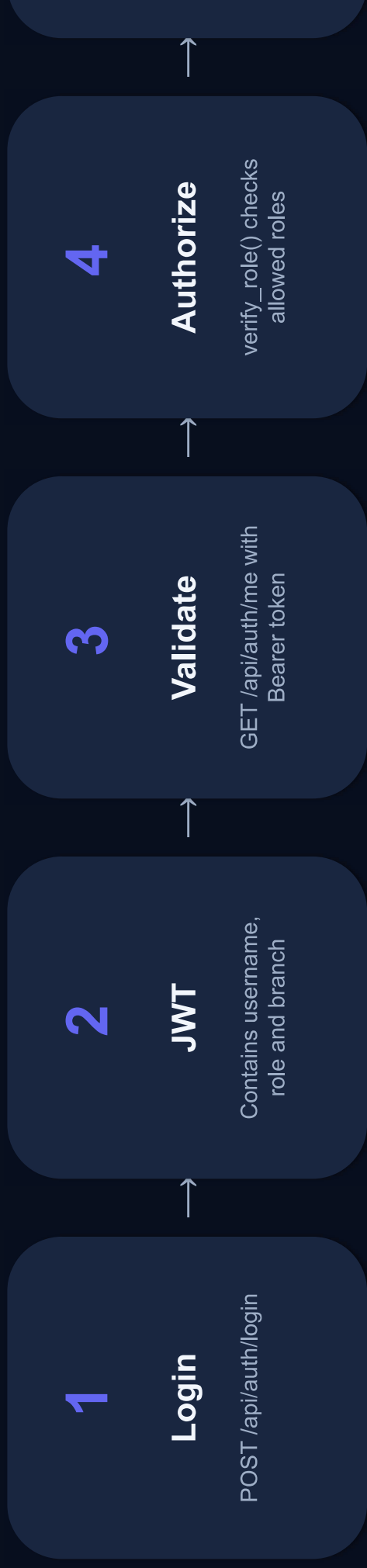
Credentials are hard-coded in backend/app/auth.py for the current project

Username	Password	Role	Scope
faculty_cse	password123	FACULTY	CSE
faculty_ece	password123	FACULTY	ECE
hod_cse	password123	HOD	CSE
hod_ece	password123	HOD	ECE
principal	password123	PRINCIPAL	All Branches

Important: these are development/demo credentials visible in the uploaded source. They should be replaced by secure user management in production use.

7. Authentication & Authorization Flow

How access control works in the code



Session behavior: the frontend currently persists cc_token in localStorage, validates it on startup, and clears it on a 401 response.



8. Faculty Dashboard

0

+ Add Student

✓ Attendance

📄 Marks Entry

📄 Quick

faculty

Register New Student

Roll No

Branch

Section

Phone

Add Student

Student Name

Semester

Email

Faculty functions

- Add new student records
- Select semester and section
- Monthly attendance entry
- Automatic attendance percentage
- Subject-wise marks entry
- Automatic row average
- Quick class summary and performers

9. Faculty Analytics & Risk Detection

What the backend calculates for a class

Total Students

120

Avg Attendance

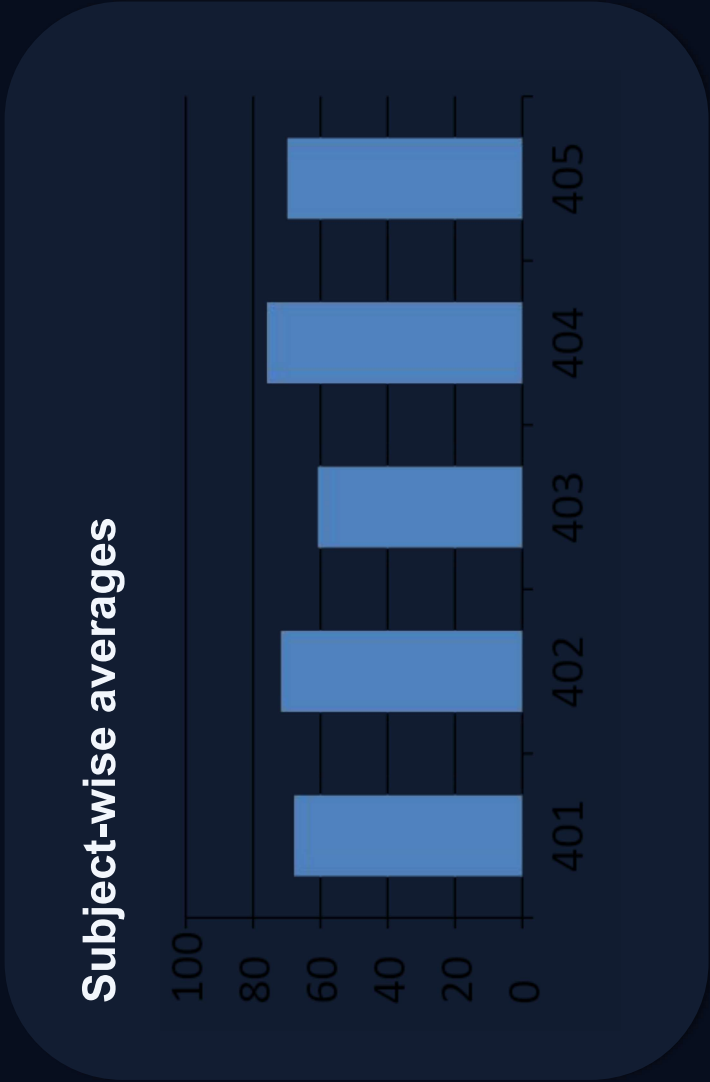
82.6%

Low Attendance

18

At-Risk Students

7



Risk rule implemented

- Low attendance list: attendance < 75%
- Top 5 students: highest recalculated average
- At Risk: attendance < 75% AND average m40%
- Subject averages are computed from the m workbook



10. HOD Dashboard

Department-level monitoring and intervention

Students

480

Avg Attendance

79.8%

High Risk

32

Medium Risk

68

Department performance

Interactive HOD features



- Semester selector: all semesters or one semester
- Attendance threshold control
- Search students by name / roll number
- Risk filtering and drill-down
- Subject-level performance details
- Department risk distribution

11. Risk Distribution Logic

Clear rules used across analytics

High Risk

Attendance < 75% AND Marks < 40%

Medium Risk

Attendance < 75% OR Marks < 40%, excluding High Risk

Satisfactory

All remaining students

These rules drive risk counts, filtering and the student lists shown to decision-makers.



12. Principal Dashboard

Campus-wide executive analytics

All Students

2,400

High Risk

156

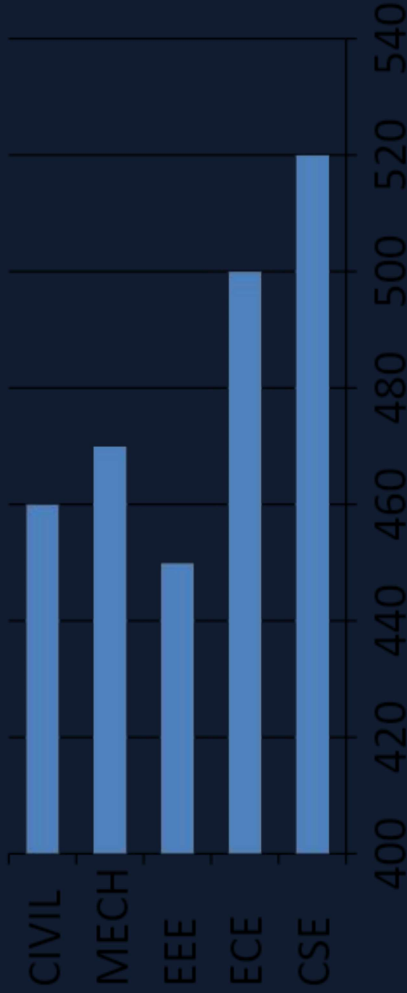
Fee Pending

~30%

Avg Attendance

80.2%

Branch comparison



Principal-only perspective

- Can access all branches rather than only assigned branch
- Campus-wide KPIs and cross-branch comparison
- Risk heatmap by branch and semester
- Drill-down to students from risk/problem
- Export selected student lists to CSV and Excel

13. Interactive Data Playground

Advanced exploration component available to analytics dashboards

Branch: All Semester: All Chart: Scatter Attendance: 0–100% Marks: 0–100 Search Reset Export

Filtered Students

2,400

Avg Attendance

80.2%

Avg Marks

71.4

High Risk

156

Chart modes

- Scatter: attendance vs marks; click points to select a student
- Histogram/buckets: student count by attendance range
- Trend: semester-wise attendance and marks

Student table

Roll No	Name	Semester	Attendance

14. Main API Surface

Endpoints found in configuration and backend

/api/auth/login	Authenticate and issue JWT	/api/analytics/principal	College-level analytics
/api/auth/me	Validate token and return current user	/api/analytics/filter	Threshold student filter
/api/faculty/add-student	Create student record	/api/analytics/risk-distribution	High / medium risk counts
/api/faculty/mark-attendance	Save monthly attendance	/api/analytics/subject	Subject performance
/api/faculty/upload-marks	Save marks and averages	/api/analytics/fee-pending	Fee-pending status
/api/analytics/faculty	Class-level analytics	/api/analytics/explore	Cross-filtered analytics
/api/analytics/hod	Department-level analytics		Cam

15. Deployment & Runtime Configuration

How the project is intended to run

Frontend

- Vite application
- Vercel deployment target
- VITE_API_URL environment variable
- Fallback: localhost:8000

Backend

- FastAPI app
- CORS currently allows all origins
- Render deployment guide included
- Python requirements file

Flow

- Browser opens frontend
- Frontend calls campusconnect API
- JWT added to production requests
- Excel data is produced by server

The deployed URL was checked as part of this analysis: CampusConnect Analytics frontend.

16. What Works Well

Strong points of the current implementation

- Clear separation of frontend components, authentication context and backend analytics modules.
- Role-based experience is explicit and easy to understand: Faculty → HOD → Principal.
- Branch-level authorization prevents Faculty/HOD users from writing into another branch.
- Excel-based structure makes the prototype easy to inspect and demo.
- Risk logic is concrete and reusable across dashboards.
- Interactive explorer supports multiple filters, chart modes, search, row selection and CSV export.
- FastAPI + Pydantic provides a clean foundation for typed API requests.

17. Critical Improvements Before Real Production Use

The no-nonsense gap between a strong prototype and a production system

Security	Move hard-coded users/passwords and SECRET_KEY to environment variables; hash passwords; do not expose demo credentials
Sessions	The frontend currently stores the JWT in localStorage; review the XSS risk and use a safer session strategy where appropriate
CORS	Replace allow_origins=["*"] with the exact approved frontend origins.
Data Layer	Excel is fine for a demo, but use PostgreSQL/MySQL for concurrent multi-user writes and auditability.
Validation	Add stricter server-side validation for marks, attendance ranges, duplicate updates and malformed months.
Testing	Add unit tests, API tests and role/branch authorization tests.
Observability	Add structured logging, error monitoring and production health checks.

18. Conclusion

CampusConnect is a complete academic analytics prototype—not just a dashboard.

- Faculty enters and maintains student, attendance and marks data.
- HOD monitors departmental performance, risk and subject trends.
- Principal sees campus-wide indicators and cross-branch issues.
- FastAPI processes Excel records and exposes protected analytics endpoints.
- React renders role-specific workflows, charts, filters and exports.

Thank You

Final assessment

Strong prototype

Best next step: harden and replace Excel with database.